Instituto Federal de Educação, Ciência e

Tecnologia de São Paulo - IFSP
Campus Jacareí
Abril de 2026
Qualidade e Teste de Software- JCRTQSW

Lista de Exercícios 08

Fabio Vivarelli
João Pedro Veríssimo
João Vitor Gimenes
Juan Santos Trigo Nasser
Nathan Henrique

Sumário

Instituto Federal de Educação, Ciência e.....1
Exercício A.....3
Exercício B.....6
Exercício C.....9
Exercício D.....12
Exercício E.....14

Exercício A

A classe ExA apresentou um excelente resultado no teste de mutação, alcançando 100% de cobertura de mutantes e 100% de força de testes, mesmo com uma cobertura de linhas de 89%. Isso indica que, apesar de nem todas as linhas terem sido executadas durante os testes, todos os mutantes gerados foram corretamente detectados e eliminados. Esse resultado demonstra que os testes implementados são altamente eficazes na validação do comportamento do método, especialmente por meio da verificação explícita das saídas geradas.

A estratégia de capturar e validar o conteúdo impresso no console foi fundamental para garantir a detecção de alterações no fluxo lógico do programa, evidenciando uma abordagem robusta de teste:

ExA.java:

```java
package com.exercicios;

public class ExA {

    public static void compute(int x, int y) {
        if (y == 0)
            System.out.println("y is zero");
        else if (x == 0)
            System.out.println("x is zero");
        else {
            for (int i = 1; i <= x; i++) {
                if (i % y == 0)
                    System.out.println(i);
            }
        }
    }

}
```

TestAll.java

```java
package com.exercicios;

import org.junit.Test;
import static org.junit.Assert.*;

public class TestAll {

    @Test
    public void testA() {
        ExA.compute(10, 2);
        ExA.compute(0, 2);
        ExA.compute(10, 0);
    }
```

ExA.java.html:

ExA.java

```
1   package com.exercicios;
2
3   public class ExA {
4
5       public static void compute(int x, int y) {
6 1         if (y == 0)
7 1             System.out.println("y is zero");
8 1         else if (x == 0)
9 1             System.out.println("x is zero");
10          else {
11 2            for (int i = 1; i <= x; i++) {
12 2                if (i % y == 0)
13 1                    System.out.println(i);
14              }
15          }
16      }
17
18  }
```

Mutations

```
6   1. negated conditional → KILLED
7   1. removed call to java/io/PrintStream::println → KILLED
8   1. negated conditional → KILLED
9   1. removed call to java/io/PrintStream::println → KILLED
11  1. changed conditional boundary → KILLED
    2. negated conditional → KILLED
12  1. Replaced integer modulus with multiplication → KILLED
    2. negated conditional → KILLED
13  1. removed call to java/io/PrintStream::println → KILLED
```

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVERT_NEGS
- MATH
- NEGATE_CONDITIONALS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- com.exercicios.TestAll.testA_divisiveis(com.exercicios.TestAll) (13 ms)
- com.exercicios.TestAll.testA_yZero(com.exercicios.TestAll) (1 ms)
- com.exercicios.TestAll.testA_xZero(com.exercicios.TestAll) (2 ms)

Report generated by PIT 1.15.0

Exercício B

A classe ExB também apresentou um desempenho bastante satisfatório, com 92% de cobertura de linhas e 92% de cobertura de mutação. Esse resultado indica que os testes conseguem cobrir a maior parte dos caminhos possíveis do algoritmo de busca binária, incluindo casos de sucesso e falha. A alta taxa de mutação demonstra que os testes são capazes de identificar alterações significativas na lógica, como mudanças nas condições de comparação e nos limites do vetor. Ainda assim, a existência de um pequeno percentual de mutantes sobreviventes sugere a possibilidade de inclusão de testes adicionais para casos extremos, como vetores vazios ou limites mais específicos, a fim de alcançar cobertura total.

```java
package com.exercicios;

public class ExB {
    public static int lookup(int vector[], int key, int n) {
        int low = 0;
        int high = n - 1;

        while (low <= high) {
            int medium = (low + high) / 2;

            if (vector[medium] == key)
                return medium;
            else if (vector[medium] < key)
                low = medium + 1;
            else
                high = medium - 1;
        }
        return -1;
    }
}
```

```java
    // ====================
    // ExB
    // ====================
    @Test
    public void testB_found_middle() {
        int v[] = {1,2,3,4,5};
```

```
        assertEquals(2, ExB.lookup(v, 3, v.length));
    }

    @Test
    public void testB_notFound() {
        int v[] = {1,2,3,4,5};
        assertEquals(-1, ExB.lookup(v, 10, v.length));
    }

    @Test
    public void testB_first() {
        int v[] = {1,2,3,4,5};
        assertEquals(0, ExB.lookup(v, 1, v.length));
    }

    @Test
    public void testB_last() {
        int v[] = {1,2,3,4,5};
        assertEquals(4, ExB.lookup(v, 5, v.length));
    }

    @Test
    public void testB_singleElement() {
        int v[] = {7};
        assertEquals(0, ExB.lookup(v, 7, v.length));
        assertEquals(-1, ExB.lookup(v, 1, v.length));
    }
```

ExB.java

```
1   package com.exercicios;
2
3   public class ExB {
4       public static int lookup(int vector[], int key, int n) {
5           int low = 0;
6 1         int high = n - 1;
7
8 2         while (low <= high) {
9 2             int medium = (low + high) / 2;
10
11 1            if (vector[medium] == key)
12 1                return medium;
13 2            else if (vector[medium] < key)
14 1                low = medium + 1;
15              else
16 1                high = medium - 1;
17          }
18 1        return -1;
19      }
20  }
```

Mutations

```
6   1. Replaced integer subtraction with addition → KILLED
8   1. negated conditional → KILLED
    2. changed conditional boundary → KILLED
9   1. Replaced integer division with multiplication → KILLED
    2. Replaced integer addition with subtraction → KILLED
11  1. negated conditional → KILLED
12  1. replaced int return with 0 for com/exercicios/ExB::lookup → KILLED
13  1. changed conditional boundary → SURVIVED
    2. negated conditional → KILLED
14  1. Replaced integer addition with subtraction → TIMED_OUT
16  1. Replaced integer subtraction with addition → TIMED_OUT
18  1. replaced int return with 0 for com/exercicios/ExB::lookup → KILLED
```

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVERT_NEGS
- MATH
- NEGATE_CONDITIONALS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- com.exercicios.TestAll.testB_first(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testB_singleElement(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testB_last(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testB_notFound(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testB_found_middle(com.exercicios.TestAll) (0 ms)

Report generated by PIT 1.15.0

Exercício C

A classe ExC apresentou um comportamento distinto das demais, com 100% de cobertura de linhas, porém apenas 45% de cobertura de mutação. Isso evidencia que, embora todos os trechos do código tenham sido executados durante os testes, uma parcela significativa dos mutantes não foi detectada. Esse resultado pode ser atribuído tanto à complexidade do método quanto à sua baixa testabilidade. O uso de saídas via System.out, aliado à ausência de valores de retorno, dificulta a verificação direta dos efeitos das mutações. Além disso, parte da lógica interna, como a manipulação de arrays, não produz efeitos observáveis externamente, permitindo que mutações sobrevivam mesmo com execução completa do código. Apesar da melhoria em relação a versões anteriores, esse resultado demonstra que a eficácia dos testes ainda é limitada por características estruturais do próprio código.

```
public class ExC {

    public void example1(boolean condition, String string1, ErrorCodes errno, char
input[]) {
        int p[] = null;
        char string2[] = null;
        String message[] = new String[3];

        if (condition) {
            p = new int[2];
            p[0] = 1;
        }

        if (p != null)
            p[1] = 0;

        if (condition)
            System.out.println("They are equal");

        message[0] = "Permission denied";
        message[1] = "No such device";
        message[2] = "No such file or directory";

        System.out.println(message[errno.getErrorNum()]);

        char output[] = new char[100];

        for (int i = 0; i < input.length && i < output.length; i++) {
            output[i] = input[i];
            if (input[i] == '\0')
                break;
        }
    }
}
```

```java
// ====================
// ExC
// ====================

@Test
public void testC_conditionTrue_printsEqual() {
    ExC c = new ExC();

    String output = captureOutput(() ->
        c.example1(true, "abc", ErrorCodes.EACCES, new char[]{'a','\0'})
    );

    assertTrue(output.contains("They are equal"));
}

@Test
public void testC_conditionFalse_noEqualPrint() {
    ExC c = new ExC();

    String output = captureOutput(() ->
        c.example1(false, "abc", ErrorCodes.EACCES, new char[]{'a','\0'})
    );

    assertFalse(output.contains("They are equal"));
}

@Test
public void testC_allErrorMessages() {
    ExC c = new ExC();

    String out1 = captureOutput(() ->
        c.example1(true, "a", ErrorCodes.EACCES, new char[]{'x','\0'})
    );
    assertTrue(out1.contains("Permission denied"));

    String out2 = captureOutput(() ->
        c.example1(true, "a", ErrorCodes.ENODEV, new char[]{'x','\0'})
    );
    assertTrue(out2.contains("No such device"));

    String out3 = captureOutput(() ->
        c.example1(true, "a", ErrorCodes.ENOENT, new char[]{'x','\0'})
    );
    assertTrue(out3.contains("No such file or directory"));
}
```

ExC.java

```
1   package com.exercicios;
2
3   enum ErrorCodes {
4       EACCES(0), ENODEV(1), ENOENT(2);
5
6       int _errorn;
7
8       ErrorCodes(int e) {
9           _errorn = e;
10      }
11
12      public int getErrorNum() {
13 1        return _errorn;
14      }
15  }
16
17  public class ExC {
18
19      public void example1(boolean condition, String string1, ErrorCodes errno, char input[]) {
20          int p[] = null;
21          char string2[] = null;
22          String message[] = new String[3];
23
24 1        if (condition) {
25              p = new int[2];
26              p[0] = 1;
27          }
28
29 1        if (p != null)
30              p[1] = 0;
31
32 1        if (condition)
33 1            System.out.println("They are equal");
34
35          message[0] = "Permission denied";
36          message[1] = "No such device";
37          message[2] = "No such file or directory";
38
39 1        System.out.println(message[errno.getErrorNum()]);
40
41          char output[] = new char[100];
42
43 4        for (int i = 0; i < input.length && i < output.length; i++) {
44              output[i] = input[i];
45 1            if (input[i] == '\0')
46                  break;
47          }
48      }
49  }
```

Mutations

```
13  1. replaced int return with 0 for com/exercicios/ErrorCodes::getErrorNum → KILLED
24  1. negated conditional → SURVIVED
29  1. negated conditional → KILLED
32  1. negated conditional → KILLED
33  1. removed call to java/io/PrintStream::println → KILLED
39  1. removed call to java/io/PrintStream::println → KILLED
43  1. negated conditional → SURVIVED
    2. negated conditional → SURVIVED
    3. changed conditional boundary → SURVIVED
    4. changed conditional boundary → SURVIVED
45  1. negated conditional → SURVIVED
```

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVERT_NEGS
- MATH
- NEGATE_CONDITIONALS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- com.exercicios.TestAll.testC_longInput(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testC_conditionTrue_printsEqual(com.exercicios.TestAll) (1 ms)
- com.exercicios.TestAll.testC_conditionFalse_noEqualPrint(com.exercicios.TestAll) (1 ms)
- com.exercicios.TestAll.testC_inputStopsAtNullChar(com.exercicios.TestAll) (1 ms)
- com.exercicios.TestAll.testC_allErrorMessages(com.exercicios.TestAll) (2 ms)

Report generated by PIT 1.15.0

Exercício D

A classe ExD apresentou resultados excelentes, atingindo 100% de cobertura de mutação e força de testes, com 89% de cobertura de linhas. Esse desempenho indica que os testes foram capazes de validar completamente a lógica do método, incluindo diferentes cenários de entrada, como valores pares, ímpares e nulos. A presença de retorno direto no método facilita a construção de testes assertivos, permitindo verificar com precisão os resultados esperados. Esse exercício exemplifica boas práticas tanto na implementação quanto na testabilidade do código, resultando em alta efetividade na detecção de mutantes.

```java
package com.exercicios;

public class ExD {
    public static Integer m(int i, Integer obj) {
        int s = 0;

        if (obj == null)
            return null;

        if (i % 2 == 1)
            i++;

        for (; i > 0; i--)
            s += obj;

        return s;
    }
}
```

```java
    @Test
    public void testD_oddBecomesEven() {
        assertEquals(Integer.valueOf(8), ExD.m(3, 2));
    }

    @Test
    public void testD_even() {
        assertEquals(Integer.valueOf(4), ExD.m(2, 2));
    }

    @Test
    public void testD_zero() {
        assertEquals(Integer.valueOf(0), ExD.m(0, 2));
    }

    @Test
    public void testD_null() {
        assertNull(ExD.m(5, null));
    }
```

ExD.java

```
1   package com.exercicios;
2
3   public class ExD {
4       public static Integer m(int i, Integer obj) {
5           int s = 0;
6
7 1         if (obj == null)
8 1             return null;
9
10 2        if (i % 2 == 1)
11 1            i++;
12
13 2        for (; i > 0; i--)
14 1            s += obj;
15
16 1        return s;
17      }
18  }
```

Mutations

7	1. negated conditional → KILLED
8	1. replaced Integer return value with 0 for com/exercicios/ExD::m → KILLED
10	1. Replaced integer modulus with multiplication → KILLED 2. negated conditional → KILLED
11	1. Changed increment from 1 to -1 → KILLED
13	1. changed conditional boundary → KILLED 2. negated conditional → KILLED
14	1. Replaced integer addition with subtraction → KILLED
16	1. replaced Integer return value with 0 for com/exercicios/ExD::m → KILLED

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVERT_NEGS
- MATH
- NEGATE_CONDITIONALS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- com.exercicios.TestAll.testD_oddBecomesEven(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testD_zero(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testD_even(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testD_null(com.exercicios.TestAll) (0 ms)

Report generated by PIT 1.15.0

Exercício E

A classe ExE apresentou resultados intermediários, com 75% de cobertura de linhas e 71% de cobertura de mutação. Isso indica que, embora os principais caminhos de execução tenham sido testados, ainda existem cenários não explorados, especialmente relacionados a condições de fronteira e combinações específicas dos parâmetros. A lógica condicional presente no método exige uma variedade maior de testes para garantir a detecção de mutantes, incluindo casos em que os valores se encontram exatamente nos limites do intervalo. Apesar disso, os testes implementados demonstram uma boa cobertura geral e uma capacidade razoável de identificar falhas, podendo ser aprimorados com a inclusão de novos cenários.

```java
package com.exercicios;

public class ExE {
    public static String contains(int x, int a, int b) {
        if (a <= b) {
            if (a <= x && x <= b)
                return "inside";

            if (x <= a || x >= b)
                return "outside";
        } else {
            return "empty";
        }
        return "none";
    }
}
```

```java
@Test
    public void testE_inside() {
        assertEquals("inside", ExE.contains(5,1,10));
    }

    @Test
    public void testE_outside_low() {
        assertEquals("outside", ExE.contains(0,1,10));
    }

    @Test
    public void testE_outside_high() {
        assertEquals("outside", ExE.contains(11,1,10));
    }

    @Test
    public void testE_empty() {
        assertEquals("empty", ExE.contains(5,10,1));
    }

    @Test
    public void testE_boundaryA() {
        assertEquals("inside", ExE.contains(1,1,10));
    }

    @Test
    public void testE_boundaryB() {
        assertEquals("inside", ExE.contains(10,1,10));
    }
}
```

ExE.java

```
1   package com.exercicios;
2
3   public class ExE {
4       public static String contains(int x, int a, int b) {
5 2         if (a <= b) {
6 4             if (a <= x && x <= b)
7 1                 return "inside";
8
9 4             if (x <= a || x >= b)
10 1                return "outside";
11          } else {
12 1            return "empty";
13          }
14 1        return "none";
15      }
16  }
```

Mutations

5	1. negated conditional → KILLED 2. changed conditional boundary → SURVIVED
6	1. changed conditional boundary → KILLED 2. changed conditional boundary → KILLED 3. negated conditional → KILLED 4. negated conditional → KILLED
7	1. replaced return value with "" for com/exercicios/ExE::contains → KILLED
9	1. negated conditional → KILLED 2. negated conditional → KILLED 3. changed conditional boundary → SURVIVED 4. changed conditional boundary → SURVIVED
10	1. replaced return value with "" for com/exercicios/ExE::contains → KILLED
12	1. replaced return value with "" for com/exercicios/ExE::contains → KILLED
14	1. replaced return value with "" for com/exercicios/ExE::contains → NO_COVERAGE

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVERT_NEGS
- MATH
- NEGATE_CONDITIONALS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- com.exercicios.TestAll.testE_empty(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testE_boundaryA(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testE_inside(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testE_boundaryB(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testE_outside_low(com.exercicios.TestAll) (0 ms)
- com.exercicios.TestAll.testE_outside_high(com.exercicios.TestAll) (0 ms)

Report generated by PIT 1.15.0